

Searching. Sorting.

Assoc. Prof. Arthur Molnar

Babeş-Bolyai University

2025

Overview

1 Searching

- The searching problem
- Searching with unordered keys
 - Sequential search
- Searching with ordered keys
 - Binary search
 - Exponential search
- Search algorithm comparison

2 Sorting

- The sorting problem
- (Binary) Insertion sort
- Quick Sort
- Merge Sort
- Introsort (introspective sort)
- TimSort
- Sorting algorithm comparison

Searching with unordered keys

Problem specification

- **Data:** A is a sequence of records with n elements, where $n \in \mathbb{N}$.
- **Result:** $index$, where $(0 \leq index \leq n - 1)$, the search key's position in sequence A , or $index = -1$, if the key was not found.

Sequential search

Sequential search (iterative and recursive)

[lecture.examples.ex09_sequential_search.py](#)^a

^aThis is a clickable link

Computational complexity:

- Best case: search key is the first element checked, so $T(n) = 1 \in \Theta(1)$
- Worst case: search key is not found, so $T(n) = n \in \Theta(n)$

Sequential search

Computational complexity:

- Average case: let's assume² 50% of searches end with the element not found, and when the element is found, it has the same probability of being on any position in the list. This time, we simulate $2 * n$ runs; during n runs, the element is not found, and during the remaining runs, it is found on each of the list's n positions;

$$T(n) = \underbrace{\frac{1 + 2 + \dots + n}{2 * n}}_{\text{key found, } n \text{ searches}} + \underbrace{\frac{n + n + \dots + n}{2 * n}}_{\text{key not found, } n \text{ searches}} \in \Theta(n)$$

²an assumption so that we may provide probability $P(I)$ 

Sequential search

Sequential search (iterative and recursive)

[lecture.examples.ex09_sequential_search.py](#)^a

^aThis is a clickable link

What about a recursive implementation³?

$$\begin{cases} T(n) = 1, n = 1 \\ T(n) = T(n-1) + 1, n \geq 1 \end{cases}$$

Leads to a telescopic sum that resolves to $T(n) \in \Theta(n)$

³Only for fun ☺, as the Python implementation call stack is capped to 1000 calls ↻ 🔍 🔍

Searching with ordered keys

Problem specification

- **Data:** A is a sequence of records with n elements, where $n \in \mathbb{N}$ so that $\forall 0 \leq i < j \leq n - 1, A[i] \leq A[j]$
- **Result:** $index$, where $(0 \leq index \leq n - 1)$, the search key's position in sequence A , or $index = -1$, if the key was not found.

Binary search

Binary search (iterative and recursive)

[lecture.examples.ex10_binary_search.py](#)^a

Pay attention to the *test_binary_search* method.

^aThis is a clickable link

Computational complexity:

- Best case: search key is the first element checked, so
 $T(n) = 1 \in \Theta(1)$

- Worst case: search key is not found.
$$\begin{cases} T(n) = 1, n = 1 \\ T(n) = T(n/2) + 1, n \geq 1 \end{cases},$$

each time halving the array in $O(1)$ time. Since we can do this $\log_2 n$ times, the complexity in the average and worst cases is $\Theta(\log_2 n)$

Exponential search

Exponential search

[lecture.examples.ex11_exponential_search.py](#)^a

Pay attention to the inclusion of the *binary_search_impl* method.

^aThis is a clickable link

Computational complexity:

We need $\log_2 n$ steps to determine whether the element is in the list; we apply further $\log_2 n$ steps to find its position between indices $i, 2 * i$ (complexity can be determined based on indices i , but for ease of understanding we used n as upper bound, so complexity is $O(\log_2 n)$)

Search algorithm comparison

Empirical comparison

[lecture.examples.ex12_search_comparison.py](#)^a

^aThis is a clickable link

The sorting problem

Sorting

Rearrange a data collection in such a way that the elements of the collection verify a given order.

Problem specification

- **Data:** A is a sequence of records with n elements, where $A = (a_1, a_2, \dots, a_n)$, $a_i \in \mathbb{R}$, $i = 1, n$
- **Results:** A' , where A' is a permutation of A , having sorted elements: $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

The sorting problem

Terminology:

- **Internal sort** - data to be sorted are available in the internal memory; in **external sorting**, data is (also) stored externally and there is a cost associated with bringing data to the internal memory
- **Streaming sort** - data becomes available sequentially (being read from the network, a database etc.)
- **In-place sort** - transforms the input data into the output, only using a small additional space. Its opposite is called out-of-place.
- **Sorting stability** - the original order of multiple records having the same key is preserved (equal elements are not switched, this is important for multi-criteria sorting)
- **Adaptive sort** - the structure of the input data is taken into account during the sorting process
- **Hybrid sort** - an algorithm that uses two or more sorting algorithms to speed up sorting

Demo

Stable sort example

[lecture.examples.ex13_stable_sort.py](#)^a

^aThis is a clickable link

Sorting algorithms

A few algorithms that we will study

- (Binary) Insertion sort
- Quick Sort
- Merge sort
- TimSort

Insertion sort

- Traverse the list once, examining each element a single time
- Insert the examined element at the right position in the subsequence of already sorted elements.
- The sub-sequence containing the already processed elements is kept sorted, so that, at the end of the traversal, the entire sequence is sorted.
- Very good for short lists, or lists that are nearly sorted (with only a few elements out of place)

Insertion sort – fact sheet

In-place	✓	Stable	✓
Streaming	✓	Space	$O(1)$
Time	Best $O(n)$	Average $O(n^2)$	Worst $O(n^2)$

Binary Insertion sort

- An optimization of insertion sort
- Since the sequence of already processed elements is sorted, we use binary search to find the place of each new element

Binary Insertion sort

Question

What happens if use (binary) insertion sort to sort **linked lists** instead of **arrays**? What are the differences?

Binary Insertion sort

Insertion sort (with binary search optimization)

[lecture/examples/ex14_insertion_sort.py](#)^a

^aThis is a clickable link

Binary Insertion sort – fact sheet

In-place	✓	Stable	✓
Streaming	✓	Space	$O(1)$
Time	Best $O(n)$	Average $O(n^2)$	Worst $O(n^2)$

Quick Sort

Based on the *divide and conquer* technique

- ① **Divide:** partition array into 2 sub-arrays such that elements in the lower part $\leq$ elements in the higher part.

Partitioning

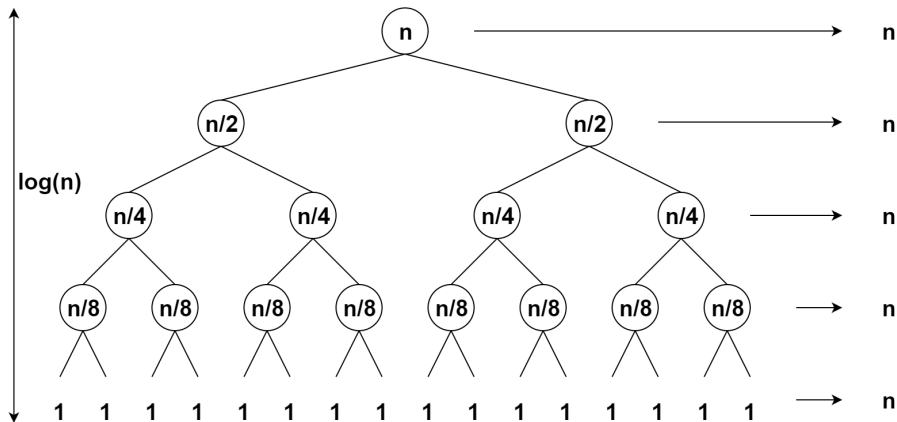
Re-arrange the elements so that the element called pivot occupies the final position in the sub-sequence. If i is that position: $k_j \leq k_i \leq k_l$, for $Left \leq j < i < l \leq Right$

- ② **Conquer:** recursively sort the 2 sub-arrays.
- ③ **Combine:** trivial since sorting is done in place.

Quick Sort - time complexity

- An example of in-place, divide & conquer, non-stable sort 😊
- Its time complexity depends on the distribution of splits, namely pivot selection. Some strategies:
 - Choose the first or last element
 - Choose a random element, or the one in the middle
 - Median of the first, middle and last elements in the partition (helps when data is already sorted or sorted in reverse)

Quick Sort - best partitioning



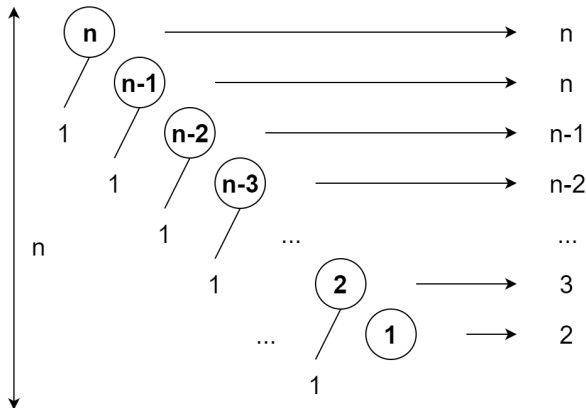
- We partition n elements $\log_2 n$ times, so $T(n) \in \Theta(n \log_2 n)$

Quick Sort - worst partitioning

- In the worst case, function Partition splits the array such that one side of the partition has only one element:

$$T(n) = T(1) + T(n-1) + \Theta(n) = T(n-1) + \Theta(n) = \sum_{k=1}^n \Theta(k) \in \Theta(n^2)$$

Quick Sort - Worst case



- Worst case appears when the input array is sorted or reverse sorted, and we select the first or last element as the pivot
- The n elements are partitioned n times, $T(n) \in \Theta(n^2)$

Quick Sort - Multi-pivot versions

- **1961** - The original version of quicksort, using a single pivot is published by Tony Hoare
- **2009** - Research shows that due to modern CPU cache architectures, the constant factor in the algorithm's complexity can be lowered by using 2 pivots

Two-pivot quicksort in Java 8

Implementation by Vladimir Yaroslavskiy (inventor of two-pivot quicksort), Jon Bentley, and Josh Bloch (author of the Java Collections Framework)
<https://hg.openjdk.org/jdk8/jdk8/jdk/file/tip/src/share/classes/java/util/DualPivotQuicksort.java>

Quick Sort - Multi-pivot versions

- **2014** - Additional research shows possible further improvements when using 3 pivots

Research paper - Multi-Pivot Quicksort: Theory and Experiments by Kushagra et al.

<https://epubs.siam.org/doi/abs/10.1137/1.9781611973198.6>

Quick Sort – fact sheet

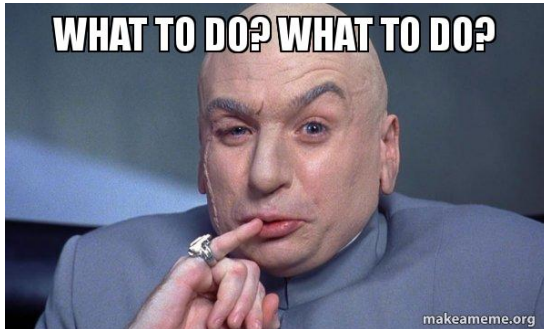
In-place	✓	Stable	X
Streaming	X	Space	$O(\log_2 n)$
Time	Best	Average	Worst
	$O(n * \log_2 n)$	$O(n * \log_2 n)$	$O(n^2)$

Merge Sort

- Based on the divide & conquer approach.
- The list to be sorted is divided in two sub-lists that are sorted separately. The sorted sub-lists are then merged.
- Each sub-list is sorted using the same approach until we get to sub-lists of length 1, which we know are sorted.

Merge Sort

- + Merge sort has good time complexity and it's easy to write an implementation that works well...
- It has non-constant extra-space complexity
- Time is wasted when merging very short lists (length 1, 2, 4, ...)



Merge Sort – fact sheet

In-place	X	Stable	X
Streaming	X	Space	$O(n)$
Time	Best	Average	Worst
	$O(n * \log_2 n)$	$O(n * \log_2 n)$	$O(n * \log_2 n)$

Sorting Algorithm Optimization

- There's no really "best" sorting algorithms, as data size, structure, cost of comparisons, CPU cache sizes and other things influence actual run times
- e.g., merge-insertion sort (a.k.a Ford-Johnson algorithm) was known for a long time as the sorting algorithm with the smallest number of element comparisons
- e.g., pivot count and selection in quicksort implementations
- Combine the strengths of several algorithms into **hybrid algorithms**, which select implementations depending on the size and structure of the input data

Sorting Algorithm Optimization

Sorting algorithms visualized

<https://www.toptal.com/developers/sorting-algorithms>

IntroSort (introspective sort)

- An optimization of quicksort's worst case - when pivot selection leads to worst case behavior
- The recursion depth is monitored by the algorithm; if a given threshold is exceeded (e.g., $2 * \log_2 n$, where n is the array length), it assumes that pivot choice is wrong and switches to heapsort⁴ and insertion sort for short sequences.
- Intro-sort is used in the .NET platform and several C/C++ implementations.
- It's a good example of a **hybrid** sorting algorithm

⁴We don't study that is it's a tree-based algorithm, and those are handled as part of a different course

TimSort

- Developed for Python in 2002 by Tim Peters
- Adopted in Java 7, Android, Swift, V8 and Rust
- Built on merge sort, it uses binary insertion sort for small sections of the list, leverages natural element ordering and in-cache data 😊
- It is both a **hybrid** as well as an **adaptive** sorting algorithm

A Word of Warning

You'll find **many** resources on the interwebs showing "TimSort", but most of them are either incomplete, or straightforward combinations of merge and insert sort. Some of them are even buggy 😞

TimSort

The Real TimSort

- The definitive description of the algorithm by its author (<https://svn.python.org/projects/python/trunk/Objects/listsort.txt>) together with its source code (<https://github.com/python/cpython/blob/main/Objects/listobject.c>)
- Some more details regarding it (<https://mail.python.org/pipermail/python-dev/2002-July/026837.html>)

TimSort

A few other resources for those interested

- Good (video) descriptions
(<https://www.awesomealgorithms.com/home/tim-sort>) and
(<https://ericmervin.medium.com/what-is-timsort-76173b49bd16>)
- Detailed, step by step description **with Python source code**
(<https://vladrisc.com/blog/2021/12/30/timsort.html>)
- Source code in Python from the RPython project (<https://github.com/reingart/pypy/blob/master/rpython/rlib/listsort.py>)

TimSort

In the author's own words...

This describes an adaptive, stable, natural mergesort, modestly called timsort (hey, I earned it 😊). It has supernatural performance on many kinds of partially ordered arrays (less than $\lg(N!)$ comparisons needed, and as few as $N-1$), yet as fast as Python's previous highly tuned samplesort hybrid on random arrays.

In a nutshell, the main routine marches over the array once, left to right, alternately identifying the next run, then merging it into the previous runs "intelligently". Everything else is complication for speed, and some hard-won measure of memory efficiency.

https:

`https://svn.python.org/projects/python/trunk/Objects/listsort.txt`

TimSort

Most existing lists include sublists that are already sorted (e.g., [1, 4, 5, 2, 99, 6, 9, 11, 10, 90] has [1, 4, 5] and [6, 9, 11] length-3 sublists)

- ① Determine the value for *minrun*, a parameter that balances good insert sort performance ($32 < \textit{minrun} < 64$) so that the list is divided into a power of 2 runs
- ② Sort consecutive runs of length *minrun* using binary insertion sort
- ③ Place runs on a stack ($O(n)$ space complexity)
- ④ Merge consecutive runs from the stack using merge sort (this leads to a stable sort)

TimSort - optimizations

- While elements are sorted, keep going even after *minrun*
- A run sorted in descending order can be reversed to ascending in $O(n)$ (check order of first two elements)
- Only call insertion sort for those parts of the run that are not already sorted
- Merging often (and soon after runs are completed) raises the chance that data is (still) in the CPU cache
- Introduce a stack invariant on the length of the merges to ensure balanced merges
- Optimize merging by finding which elements remain in the existing order (e.g., best case when the first element of run A is larger than the last of run B)
- When merging uses data from the same run repeatedly, use an exponential search (galloping) to identify how many more elements can be used

TimSort – fact sheet

In-place	✓	Stable	✓
Streaming	X	Space	$O(n)$
Time	Best $O(n)$	Average $O(n * \log_2 n)$	Worst $O(n * \log_2 n)$

Sorting algorithm comparison

Empirical comparison

[lecture.examples.ex18_sort_comparison.py](#)^a

^aProTip: This is a clickable link